

LAPORAN PRAKTIKUM
STRUKTUR DATA
“SINGLE LINKED LIST”



Disusun oleh:

Ndaru Pradana Gatot Suseno
2411532007

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena Laporan Praktikum Struktur Data 5 dapat diselesaikan. Laporan ini disusun untuk mendokumentasikan penerapan Single Linked List dalam bahasa Java.

Praktikum kelima membahas pembentukan node, penelusuran, penambahan, pencarian, dan penghapusan pada Single Linked List. Pada laporan tugas, struktur tersebut digunakan untuk membangun sistem antrian pasien rumah sakit yang menyimpan nomor antrean, nama, dan keluhan pasien.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas arahan selama kegiatan. Pembahasan mengacu pada source code package pekan5_2411532007 dan hasil pengujian program. Penulis terbuka terhadap kritik dan saran untuk penyempurnaan laporan.

Padang, 2026

Ndaru Pradana G.S

DAFTAR ISI

KATA PENGANTAR	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
Latar Belakang	4
Tujuan Praktikum.....	4
Manfaat Praktikum.....	4
BAB II PEMBAHASAN	5
Dasar Teori.....	5
Program NodeSLL_2411532007 dan TambahSLL_2411532007	6
Program PencarianSLL_2411532007	9
Program HapusSLL_2411532007.....	10
BAB III KESIMPULAN.....	12
DAFTAR PUSTAKA	13
LAPORAN TUGAS	14
Tujuan Tugas.....	14
Class Pasien_2411532007.....	15
Class RumahSakit_2411532007	16
Hasil Eksekusi Laporan Tugas.....	19
Analisis Hasil	20
Kesimpulan Laporan Tugas	21
Saran.....	21

BAB I

PENDAHULUAN

Latar Belakang

Single Linked List merupakan struktur data dinamis yang tersusun atas node. Setiap node menyimpan data dan satu referensi next yang menunjuk ke node berikutnya. Node pertama disebut head, sedangkan node terakhir memiliki nilai next null.

Berbeda dari array, linked list tidak memerlukan alokasi elemen secara berurutan di memori. Penambahan dan penghapusan dilakukan dengan mengubah hubungan pointer. Praktikum ini mempelajari operasi tersebut dan menerapkannya pada daftar pasien yang bertambah serta berkurang selama program berjalan.

Tujuan Praktikum

1. Memahami komponen data dan next pada node SLL.
2. Melakukan traversal dan pencarian nilai.
3. Menambahkan node di awal, akhir, dan posisi tertentu.
4. Menghapus head, node terakhir, dan node berdasarkan posisi.
5. Menerapkan SLL untuk sistem antrian pasien rumah sakit.

Manfaat Praktikum

Praktikum ini membangun pemahaman tentang struktur data dinamis berbasis referensi, pengelolaan head, serta perubahan hubungan antar-node. Kompetensi tersebut menjadi dasar untuk mempelajari variasi linked list dan struktur yang menggunakan node.

BAB II

PEMBAHASAN

Dasar Teori

Node SLL terdiri atas bagian data dan referensi next. Traversal dimulai dari head dan bergerak mengikuti next sampai null. Pencarian dilakukan dengan membandingkan data pada setiap node.

Penambahan di depan menghubungkan node baru ke head lama dan mengganti head. Penambahan di akhir membutuhkan traversal ke node terakhir. Penghapusan dilakukan dengan memindahkan head atau menghubungkan node sebelumnya ke node setelah target.

Program NodeSLL_2411532007 dan TambahSLL_2411532007

NodeSLL_2411532007 mendefinisikan data integer dan pointer next. TambahSLL_2411532007 menyediakan operasi penambahan di depan, akhir, dan posisi tertentu.

Source Code NodeSLL_2411532007

```
1 package pekan5_2411532007;
2
3 public class NodeSLL_2411532007 {
4
5     // node bagian data
6     int data_2007;
7     // pointer ke node berikutnya
8     NodeSLL_2411532007 next_2007;
9     // konstruktor menginisialisasi node dengan data
10    public NodeSLL_2411532007(int data_2007) {
11        this.data_2007 = data_2007;
12        this.next_2007 = null;
13    }
14
15 }
16
17 }
```

Source Code TambahSLL_2411532007

```

1 package pekan5_2411532007;
2
3 public class TambahSLL_2411532007 {
4
5     public static NodeSLL_2411532007 insertAtFront_2007(NodeSLL_2411532007 head_2007, int value_2007) {
6         NodeSLL_2411532007 new_node_2007 = new NodeSLL_2411532007(value_2007);
7         new_node_2007.next_2007 = head_2007;
8         return new_node_2007;
9     }
10    // fungsi menambahkan node di akhir SLL
11    public static NodeSLL_2411532007 insertAtEnd_2007(NodeSLL_2411532007 head_2007, int value_2007) {
12        // buat sebuah node dengan sebuah nilai
13        NodeSLL_2411532007 newNode_2007 = new NodeSLL_2411532007(value_2007);
14        // jika list kosong maka node jadi head
15        if (head_2007 == null) {
16            return newNode_2007;
17        }
18        // simpan head ke variabel sementara
19        NodeSLL_2411532007 last_2007 = head_2007;
20        // telusuri ke node akhir
21        while (last_2007.next_2007 != null) {
22            last_2007 = last_2007.next_2007;
23        }
24        // ubah pointer
25        last_2007.next_2007 = newNode_2007;
26        return head_2007;
27    }
28
29    static NodeSLL_2411532007 GetNode_2007(int data_2007) {
30        return new NodeSLL_2411532007(data_2007);
31    }
32
33    static NodeSLL_2411532007 insertPos_2007(NodeSLL_2411532007 headNode_2007, int position_2007, int value_2007) {
34        NodeSLL_2411532007 head_2007 = headNode_2007;
35        if (position_2007 < 1) {
36            System.out.print("Invalid position");
37        }
38        if (position_2007 == 1) {
39            NodeSLL_2411532007 new_node_2007 = new NodeSLL_2411532007(value_2007);
40            new_node_2007.next_2007 = head_2007;
41            return new_node_2007;
42        } else {
43            while (position_2007-- != 0) {
44                while (position_2007-- != 0) {
45                    if (position_2007 == 1) {
46                        NodeSLL_2411532007 newNode_2007 = GetNode_2007(value_2007);
47                        newNode_2007.next_2007 = headNode_2007.next_2007;
48                        headNode_2007.next_2007 = newNode_2007;
49                        break;
50                    }
51                    headNode_2007 = headNode_2007.next_2007;
52                }
53                if (position_2007 != 1)
54                    System.out.print("Posisi di luar jangkauan");
55            }
56            return head_2007;
57        }
58    }
59    public static void printList_2007(NodeSLL_2411532007 head_2007) {
60        NodeSLL_2411532007 curr_2007 = head_2007;
61        while (curr_2007.next_2007 != null) {
62            System.out.print(curr_2007.data_2007 + "-->");
63            curr_2007 = curr_2007.next_2007;
64        }
65        if (curr_2007.next_2007 == null) {
66            System.out.print(curr_2007.data_2007);
67        }
68        System.out.println();
69    }
70    public static void main (String[] args) {
71        // buat linked list 2->3->5->6
72        NodeSLL_2411532007 head_2007 = new NodeSLL_2411532007(2);
73        head_2007.next_2007 = new NodeSLL_2411532007(3);
74        head_2007.next_2007.next_2007 = new NodeSLL_2411532007(5);
75        head_2007.next_2007.next_2007.next_2007 = new NodeSLL_2411532007(6);
76
77        // cetak list asli
78        System.out.print("Senarai berantai awal:");
79        printList_2007(head_2007);
80
81        // tambahkan node baru di depan
82        System.out.print("tambah 1 simpul di depan: ");
83        int data_2007 = 1;
84        head_2007 = insertAtFront_2007(head_2007, data_2007);
85    }

```

```

85     head_2007 = insertFront_2007(head_2007, data_2007);
86     // cetak update list
87     printList_2007(head_2007);
88
89     // tambahkan node baru di belakang
90     System.out.print("tambah 1 simpul di belakang: ");
91     int data2_2007 = 7;
92     head_2007 = insertAtEnd_2007(head_2007, data2_2007);
93
94     // cetak update list
95     printList_2007(head_2007);
96     System.out.print("tambah 1 simpul ke data 4: ");
97     int data3_2007 = 4;
98     int pos_2007 = 4;
99     head_2007 = insertPos_2007(head_2007, pos_2007, data3_2007);
100
101     // cetak update list
102     printList_2007(head_2007);
103 }
104 }
105 }
106 }
107 }

```

Penjelasan Operasi

1. insertAtFront_2007() menjadikan node baru sebagai head.
2. insertAtEnd_2007() menelusuri node terakhir lalu menghubungkan node baru.
3. insertPos_2007() menyisipkan node pada posisi yang ditentukan.
4. printList_2007() menampilkan data dari head sampai node terakhir.

Hasil Eksekusi

```

Senarai berantai awal:2->3->5->6
tambah 1 simpul di depan: 1->2->3->5->6
tambah 1 simpul di belakang: 1->2->3->5->6->7
tambah 1 simpul ke data 4: 1->2->3->4->5->6->7

```

Analisis

List awal 2→3→5→6 berubah menjadi 1→2→3→5→6 setelah penambahan di depan. Nilai 7 kemudian ditambahkan di akhir, dan nilai 4 disisipkan pada posisi keempat.

Hasil akhir 1→2→3→4→5→6→7 menunjukkan bahwa hubungan next diperbarui dengan benar pada ketiga jenis penambahan.

Program PencarianSLL_2411532007

Program membentuk SLL berisi 14, 21, 13, 30, dan 10, kemudian menelusuri list untuk mencari key 30.

Source Code PencarianSLL_2411532007

```

1 package pekan5_2411532007;
2
3 public class PencarianSLL_2411532007 {
4     static boolean searchKey_2007(NodeSLL_2411532007 head_2007, int key_2007) {
5         NodeSLL_2411532007 curr_2007 = head_2007;
6         while (curr_2007 != null) {
7             if (curr_2007.data_2007 == key_2007)
8                 return true;
9             curr_2007 = curr_2007.next_2007;
10        }
11        return false;
12    }
13
14    public static void traversal_2007(NodeSLL_2411532007 head_2007) {
15        // mulai dari head
16        NodeSLL_2411532007 curr_2007 = head_2007;
17        while (curr_2007 != null) {
18            System.out.print(" " + curr_2007.data_2007);
19            curr_2007 = curr_2007.next_2007;
20        }
21        System.out.println();
22    }
23
24    public static void main(String[] args) {
25        NodeSLL_2411532007 head_2007 = new NodeSLL_2411532007(14);
26        head_2007.next_2007 = new NodeSLL_2411532007(21);
27        head_2007.next_2007.next_2007 = new NodeSLL_2411532007(13);
28        head_2007.next_2007.next_2007.next_2007 = new NodeSLL_2411532007(30);
29        head_2007.next_2007.next_2007.next_2007.next_2007 = new NodeSLL_2411532007(10);
30        System.out.print("Penelusuran SLL : ");
31        traversal_2007(head_2007);
32        // data yang akan dicari
33        int key_2007 = 30;
34        System.out.print("cari data " + key_2007 + " = ");
35        if (searchKey_2007(head_2007, key_2007))
36            System.out.println("ketemu");
37        else
38            System.out.println("tidak ada");
39    }
40 }
41 }
42

```

Hasil Eksekusi

```

Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu

```

Analisis

Method searchKey_2007 memulai pemeriksaan dari head dan berhenti ketika data sama dengan key. Nilai 30 ditemukan pada node keempat sehingga method mengembalikan true.

Jika traversal mencapai null tanpa kecocokan, method mengembalikan false. Kompleksitas waktu pencarian terburuk adalah $O(n)$.

Program HapusSLL_2411532007

Program ini menguji penghapusan head, node terakhir, dan node pada posisi tertentu dari list berisi 1 sampai 6.

Source Code HapusSLL_2411532007

```

1 package pekan5_2411532007;
2
3 public class HapusSLL_2411532007 {
4     // fungsi untuk menghapus head
5     public static NodeSLL_2411532007 deleteHead_2007(NodeSLL_2411532007 head_2007) {
6         // jika SLL kosong
7         if (head_2007 == null)
8             return null;
9         // pindahkan head ke node berikutnya
10        head_2007 = head_2007.next_2007;
11        // return head baru
12        return head_2007;
13    }
14    // fungsi menghapus node terakhir SLL
15    public static NodeSLL_2411532007 removeLastNode_2007(NodeSLL_2411532007 head_2007) {
16        // jika list kosong, return null
17        if (head_2007 == null) {
18            return null;
19        }
20        // jika list satu node, hapus node dan return null
21        if (head_2007.next_2007 == null) {
22            return null;
23        }
24        // temukan node terakhir ke dua
25        NodeSLL_2411532007 secondLast_2007 = head_2007;
26        while (secondLast_2007.next_2007.next_2007 != null) {
27            secondLast_2007 = secondLast_2007.next_2007;
28        }
29        // hapus node terakhir
30        secondLast_2007.next_2007 = null;
31        return head_2007;
32    }
33    // fungsi menghapus node di posisi tertentu
34    public static NodeSLL_2411532007 deleteNode_2007(NodeSLL_2411532007 head_2007, int position_2007) {
35        NodeSLL_2411532007 temp_2007 = head_2007;
36        NodeSLL_2411532007 prev_2007 = null;
37        // jika linked list null
38        if (temp_2007 == null)
39            return head_2007;
40        // kasus 1: head dihapus
41        if (position_2007 == 1) {
42            head_2007 = temp_2007.next_2007;
43            return head_2007;
44        }
45        // kasus 2: menghapus node di tengah
46        // telusuri ke node yang di hapus
47        for (int i_2007 = 1; temp_2007 != null && i_2007 < position_2007; i_2007++) {
48            prev_2007 = temp_2007;
49            temp_2007 = temp_2007.next_2007;
50        }
51        // jika ditemukan, hapus node
52        if (temp_2007 != null) {
53            prev_2007.next_2007 = temp_2007.next_2007;
54        } else {
55            System.out.println("Data tidak ada");
56        }
57        return head_2007;
58    }
59    // fungsi mencetak SLL
60    public static void printlist_2007(NodeSLL_2411532007 head_2007) {
61        NodeSLL_2411532007 curr_2007 = head_2007;
62        while (curr_2007.next_2007 != null) {
63            System.out.print(curr_2007.data_2007+"->");
64            curr_2007 = curr_2007.next_2007;
65        }
66        if (curr_2007.next_2007 == null) {
67            System.out.print(curr_2007.data_2007);
68        }
69        System.out.println();
70    }
71    // kelas main
72    public static void main(String[] args) {
73        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
74        NodeSLL_2411532007 head_2007 = new NodeSLL_2411532007(1);
75        head_2007.next_2007 = new NodeSLL_2411532007(2);
76        head_2007.next_2007.next_2007 = new NodeSLL_2411532007(3);
77        head_2007.next_2007.next_2007.next_2007 = new NodeSLL_2411532007(4);
78        head_2007.next_2007.next_2007.next_2007.next_2007 = new NodeSLL_2411532007(5);
79        head_2007.next_2007.next_2007.next_2007.next_2007.next_2007 = new NodeSLL_2411532007(6);
80        // cetak list awal
81        System.out.println("list awal: ");
82        printlist_2007(head_2007);
83        //hapus head
84    }
85

```

```

83     printList_2007(head_2007);
84
85     //hapus head
86     head_2007 = deleteHead_2007(head_2007);
87     System.out.println("List setelah head dihapus: ");
88     printList_2007(head_2007);
89
90     //hapus node terakhir
91     head_2007 = removeLastNode_2007(head_2007);
92     System.out.println("List setelah simpul terakhir di hapus: ");
93     printList_2007(head_2007);
94
95     // deleting node at position 2
96     int position_2007 = 2;
97     head_2007 = deleteNode_2007(head_2007, position_2007);
98
99     // print list after deletion
100    System.out.println("List setelah posisi 2 dihapus: ");
101    printList_2007(head_2007);
102 }
103 }
104

```

Penjelasan Operasi

1. deleteHead_2007() memindahkan head ke node berikutnya.
2. removeLastNode_2007() mencari node kedua terakhir lalu mengatur next menjadi null.
3. deleteNode_2007() menghubungkan node sebelumnya ke node setelah target.

Hasil Eksekusi

```

<terminated> HapusSLI_2411532007 [Java Application] C:\Users\ndaru\p2\pool\plugins\org.eclipse.justi.openjdkhotspot.jre.full.win32.x86_64_23.0.1.v20241024-1700\jre\bin\javaw.exe (4 Jul 2
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

```

Analisis

Setelah head dihapus, list dimulai dari 2. Penghapusan node terakhir menghilangkan nilai 6. Penghapusan posisi kedua selanjutnya menghilangkan nilai 3 sehingga list akhir menjadi 2→4→5.

Operasi memperlihatkan bahwa node dapat dihapus tanpa menggeser data lain; perubahan cukup dilakukan pada referensi head atau next.

BAB III

KESIMPULAN

Praktikum 5 berhasil menerapkan pembentukan, traversal, pencarian, penambahan, dan penghapusan node pada Single Linked List. Hubungan next menjadi unsur utama yang menentukan urutan data.

SLL memberikan ukuran dinamis dan memungkinkan perubahan struktur melalui referensi. Implementasi antrian rumah sakit menunjukkan bahwa node dapat menyimpan data objek yang lebih kompleks daripada integer.

DAFTAR PUSTAKA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Sierra, K., Bates, B., & Gee, T. (2022). Head First Java (3rd ed.). O'Reilly Media.

Oracle. (2024). Java Platform, Standard Edition API Specification. Oracle Corporation.

LAPORAN TUGAS

Laporan tugas Praktikum 5 mengembangkan sistem antrian rumah sakit menggunakan class Pasien_2411532007 sebagai node dan RumahSakit_2411532007 sebagai pengelola list sekaligus driver interaktif.

Tujuan Tugas

1. Menyimpan nomor antrean, nama, dan keluhan pada node pasien.
2. Mendaftarkan pasien di akhir SLL.
3. Memanggil pasien dari head.
4. Menampilkan, mencari, dan memeriksa status antrean.
5. Menguji perubahan daftar pasien melalui menu.

Class Pasien_2411532007

Class Pasien_2411532007 berperan sebagai node SLL. Selain atribut data pasien, class ini mempunyai referensi next_2007 untuk menunjuk pasien berikutnya.

Source Code

```

1 package pekan5_2411532007;
2
3 public class Pasien_2411532007 {
4     String nama_2007;
5     String keluhan_2007;
6     int Antrian_2007;
7
8     Pasien_2411532007 next_2007;
9
10    public Pasien_2411532007(int a, String n, String k) {
11        this.Antrian_2007 = a;
12        this.nama_2007 = n;
13        this.keluhan_2007 = k;
14        this.next_2007 = null;
15    }
16
17    public void setN_2007( String n){
18        this.nama_2007 = n;
19    }
20
21    public void setK_2007( String k){
22        this.keluhan_2007 = k;
23    }
24
25    public void setA_2007( int a){
26        this.Antrian_2007 = a;
27    }
28
29    public String getN_2007() {
30        return nama_2007;
31    }
32
33    public String getK_2007() {
34        return keluhan_2007;
35    }
36
37    public int getA_2007() {
38        return Antrian_2007;
39    }
40
41 }
42

```

Penjelasan

1. Antrian_2007 menyimpan nomor antrean otomatis.
2. nama_2007 dan keluhan_2007 menyimpan identitas serta keluhan pasien.
3. next_2007 menghubungkan node pasien dengan node berikutnya.
4. Setter dan getter menyediakan akses terhadap atribut data.

Class RumahSakit_2411532007

Class RumahSakit_2411532007 mengatur pendaftaran, pemanggilan, penampilan, pencarian, dan status antrian. Variabel item_2007 pada main menyimpan head dari SLL pasien.

Source Code

```

1 package pekan5_2411532007;
2
3 import java.util.Scanner;
4
5
6 public class RumahSakit_2411532007 {
7     private static int count_2007 = 0;
8
9     public static Pasien_2411532007 DaftarkanPasien_2007(Pasien_2411532007 item_2007, String n_2007, String k_2007 ) {
10         count_2007++;
11         Pasien_2411532007 node_2007 = new Pasien_2411532007(count_2007, n_2007, k_2007 );
12
13         if (item_2007 == null ) {
14             return node_2007;
15         }
16
17         Pasien_2411532007 L_2007 = item_2007;
18
19         while (L_2007.next_2007 != null) {
20             L_2007 = L_2007.next_2007;
21         }
22
23         L_2007.next_2007 = node_2007;
24         return item_2007;
25     }
26
27     public static Pasien_2411532007 PanggilPasien_2007(Pasien_2411532007 item_2007) {
28         if (item_2007 == null ) {
29             return null;
30         }
31
32         System.out.println("Dipanggil Pasien");
33         System.out.println("No Antrian: " + item_2007.getA_2007());
34         System.out.println("Nama: " + item_2007.getN_2007());
35         System.out.println("Keluhan/Penyakit: " + item_2007.getK_2007());
36         System.out.println("");
37
38         item_2007 = item_2007.next_2007;
39         return item_2007;
40     }
41
42     public static void TampilkanAntrian_2007(Pasien_2411532007 item_2007) {
43         Pasien_2411532007 c_2007 = item_2007;
44         Pasien_2411532007 c_2007 = item_2007;
45         int posisi_2007 = 1;
46
47         while (c_2007 != null) {
48             System.out.println("Posisi " + posisi_2007);
49             System.out.println("No Antrian: " + c_2007.getA_2007());
50             System.out.println("Nama: " + c_2007.getN_2007());
51             System.out.println("Keluhan/Penyakit: " + c_2007.getK_2007());
52             System.out.println("");
53             c_2007 = c_2007.next_2007;
54             posisi_2007++;
55         }
56     }
57
58     public static boolean CariPasien_2007(Pasien_2411532007 item_2007, String n_2007) {
59         Pasien_2411532007 c_2007 = item_2007;
60         while (c_2007 != null) {
61             if (c_2007.nama_2007.equalsIgnoreCase(n_2007)) {
62                 System.out.println("No Antrian: " + c_2007.getA_2007());
63                 System.out.println("Nama: " + c_2007.getN_2007());
64                 System.out.println("Keluhan/Penyakit: " + c_2007.getK_2007());
65                 System.out.println("");
66                 return true;
67             }
68             c_2007 = c_2007.next_2007;
69         }
70         System.out.println("tidak ada pasien dengan nama yang kamu inputkan");
71         return false;
72     }
73
74     public static void CekStatusAntrian_2007(Pasien_2411532007 item_2007) {
75         if ( item_2007 == null) {
76             System.out.println("Antrian Kosong");
77             return;
78         }
79
80         int ttl_2007 = 0;
81         Pasien_2411532007 c_2007 = item_2007;
82         while (c_2007 != null) {
83             ttl_2007++;
84             c_2007 = c_2007.next_2007;
85         }

```



```

85     }
86
87     System.out.println("Jumlah Pasien: " + ttl_2007);
88     System.out.println("Pasien Terdepan");
89     System.out.println("No Antrian: " + item_2007.getA_2007());
90     System.out.println("Nama: " + item_2007.getN_2007());
91     System.out.println("Keluhan/Penyakit: " + item_2007.getK_2007());
92     System.out.println("");
93 }
94
95
96
97 public static void main(String[] args) {
98     Scanner scanner_2007 = new Scanner(System.in);
99     Pasien_2411532007 o_2007 = null;
100
101     int pilihan_2007 = 0;
102
103     do {
104         System.out.println("\n== Antrian Rumah Sakit Nim:2411532007 ==");
105         System.out.println("1. Daftarkan Pasien");
106         System.out.println("2. Panggil Pasien");
107         System.out.println("3. Tampilkan Antrian");
108         System.out.println("4. Cari Pasien");
109         System.out.println("5. Cek Status Antrian");
110         System.out.println("6. Keluar");
111         System.out.print("Pilihan: ");
112
113         if (scanner_2007.hasNextInt()) {
114             pilihan_2007 = scanner_2007.nextInt();
115             scanner_2007.nextLine();
116         } else {
117             System.out.println("Input tidak valid. Masukkan angka.");
118             scanner_2007.nextLine();
119             continue;
120         }
121
122         switch (pilihan_2007) {
123             case 1:
124                 System.out.print("Masukkan nama pasien: ");
125                 String nama_2007 = scanner_2007.nextLine();
126                 System.out.print("Masukkan keluhan: ");
127                 String penyakit_2007 = scanner_2007.nextLine();
128
129                 o_2007 = DaftarkanPasien_2007(o_2007, nama_2007, penyakit_2007);
130                 System.out.print("Pasien berhasil didaftarkan dengan no. antrian: " + count_2007 + "\n");
131                 //
132                 break;
133             case 2:
134                 o_2007 = PanggilPasien_2007(o_2007);
135                 break;
136             case 3:
137                 TampilkanAntrian_2007(o_2007);
138                 break;
139             case 4:
140                 System.out.print("Masukkan nama pasien yang dicari: ");
141                 String n_2007 = scanner_2007.nextLine();
142                 CariPasien_2007(o_2007, n_2007);
143                 break;
144             case 5:
145                 CekStatusAntrian_2007(o_2007);
146                 break;
147             case 6:
148                 System.out.println("Program selesai");
149                 break;
150             default:
151                 System.out.println("Pilihan tidak valid.");
152                 break;
153         }
154     } while (pilihan_2007 != 6);
155     scanner_2007.close();
156 }
157
158
159
160
161
162
163
164
165

```

Penjelasan

1. DaftarkanPasien_2007() menambah pasien baru pada akhir list dan memberikan nomor antrean.
2. PanggilPasien_2007() menampilkan pasien head lalu memindahkan head ke next.
3. TampilkanAntrian_2007() menelusuri seluruh pasien.
4. CariPasien_2007() mencari nama tanpa membedakan huruf besar-kecil.

5. CekStatusAntrian_2007() menghitung jumlah pasien dan menampilkan pasien terdepan.

Hasil Eksekusi Laporan Tugas

```
=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan nama pasien: andi
Masukkan keluhan: demam
Pasien berhasil didaftarkan dengan no. antrian: 1

=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan nama pasien: budi
Masukkan keluhan: batuk
Pasien berhasil didaftarkan dengan no. antrian: 2

=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama pasien yang dicari: budi
No Antrian: 2
Nama: budi
Keluhan/Penyakit: batuk

=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
```

```

=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Posisi 1
No Antrian: 1
Nama: andi
Keluhan/Penyakit: demam

Posisi 2
No Antrian: 2
Nama: budi
Keluhan/Penyakit: batuk

=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Dipanggil Pasien
No Antrian: 1
Nama: andi
Keluhan/Penyakit: demam

=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Dipanggil Pasien
No Antrian: 2
Nama: budi
Keluhan/Penyakit: batuk

=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2

=== Antrian Rumah Sakit Nim:2411532007 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai

```

Analisis Hasil

Andi dan Budi didaftarkan dengan nomor antrean 1 dan 2. Penampilan list menunjukkan keduanya sesuai urutan pendaftaran. Pencarian nama Budi menemukan node kedua dan menampilkan keluhannya.

Status antrean menunjukkan dua pasien dengan Andi sebagai pasien terdepan. Setelah Andi dipanggil, head berpindah ke Budi dan daftar berikutnya hanya memuat satu pasien.

Kesimpulan Laporan Tugas

Class Pasien_2411532007 dan RumahSakit_2411532007 berhasil menerapkan SLL pada antrian pasien. Penambahan dilakukan di akhir, sedangkan pemanggilan pasien menghapus node pada head sesuai urutan pelayanan.

Saran

Program dapat ditambah dengan prioritas pasien darurat, penyimpanan waktu pendaftaran, penghapusan pasien tertentu, validasi data kosong, dan penyimpanan permanen ke basis data atau file.